



Instituto Tecnológico de Puerto Rico Recinto de San Juan

DOCUMENTACIÓN DEL PROGRAMA PERSONAL

Versiones 1.0, 2.0 y 3.0

Plantilla para acompañar el Laboratorio 3 - Semana 4

Estudiante	Juan Antonio Reyes Fernandez
Repositorio	TESI2105_Programa_Evolutivo_ReyesJuan
Nombre del programa	BoriTech
Fecha	9 de septiembre de 206

1. DESCRIPCIÓN GENERAL Y CONTINUIDAD

Describe brevemente la necesidad que atiende tu programa y explica cómo las versiones anteriores preparan el punto de partida para v3.0. No repitas las respuestas del documento del laboratorio; resume la continuidad en 4 a 6 oraciones.

El programa BoriTech surge para automatizar y agilizar el control de inventario de equipos electrónicos, calculando impuestos y descuentos de forma rápida y precisa. Todo comenzó con la versión 1.0, donde se creó la lógica inicial para registrar un solo equipo con su respectivo cálculo de IVU. Luego, la versión 2.0 mejoró el sistema al añadir un ciclo para ingresar múltiples productos, validar que los datos fueran correctos y aplicar un descuento por compras grandes. Estas dos versiones permitieron probar y perfeccionar el funcionamiento del programa en casos reales. Con esa base ya lista, la versión 3.0 da el paso final al organizar todo el código en funciones modulares. De esta manera, el sistema no solo funciona correctamente, sino que ahora es mucho más ordenado, fácil de entender y sencillo de mantener en el tiempo.

2. RESUMEN DE LAS VERSIONES ANTERIORES

Versión	Base técnica	Cómo se refleja en mi programa
v1.0	Programa secuencial con entradas, variables, constantes, operadores y salidas.	Permite registrar un solo equipo electrónico, pidiendo el nombre, precio y cantidad para calcular el subtotal, el impuesto del IVU (11.5%) y el costo total en una sola corrida.
v2.0	El mismo programa incorpora decisiones y ciclos.	Permite registrar varios equipos seguidos mediante un ciclo <code>while</code> , valida que el precio y cantidad sean mayores a cero y aplica un 5% de descuento si la compra supera los \$500.

Este resumen no sustituye la documentación entregada en la Semana 3; únicamente establece continuidad para la nueva versión.

3. VERSIÓN 3.0 - ORGANIZACIÓN MEDIANTE FUNCIONES

Explica en 3 a 5 oraciones cómo reorganizaste el código de v2.0 para crear v3.0 mediante funciones con responsabilidades claras.

Para la versión 3.0, dividí el código largo de la v2.0 en bloques más pequeños y específicos llamados funciones. Creé `solicitar_datos_equipo()` solo para pedir la información al usuario, `calcular_totales_inventario()` para encargarse de las matemáticas del descuento e IVU, y `mostrar_resumen()` únicamente para imprimir el recibo final. Finalmente, uní todo dentro de una función principal llamada `procesar_registro()` que controla la repetición del programa. De esta forma, cada parte del código tiene una sola tarea clara, lo que hace que el programa sea mucho más fácil de leer y arreglar en el futuro.

4. FUNCIONES CREADAS EN V3.0

Completa una fila por cada función creada. Debes documentar al menos tres funciones.

Función	Propósito	Parámetros	Retorno o acción
<code>solicitar_datos_equipo()</code>	Pedir al usuario el nombre, precio y cantidad del equipo electrónico.	N/A	Devuelve los tres valores ingresados: <code>nombre</code> , <code>precio</code> y <code>cantidad</code>
<code>calcular_totales_inventario()</code>	Calcular si aplica un 5% de descuento, sacar el subtotal con descuento, el impuesto del IVU y el costo total abonado.	<code>subtotal</code> , <code>impuesto_rate</code>	Devuelve los cuatro resultados: <code>descuento</code> , <code>subtotal_con_descuento</code> , <code>monto_impuesto</code> y <code>costo_total</code>
<code>mostrar_resumen()</code>	Imprimir en pantalla el recibo ordenado con toda la información y totales del equipo registrado.	<code>nombre</code> , <code>cantidad</code> , <code>precio</code> , <code>subtotal</code> , <code>descuento</code> , <code>monto_impuesto</code> , <code>costo_total</code>	No devuelve nada (<code>None</code>); su acción es mostrar el resumen completo en la pantalla.

5. FUNCIÓN INCORPORADA DE PYTHON

Función incorporada (built-in): `input()`

Dónde se utiliza: Dentro de la función `solicitar_datos_equipo()` para pedir el nombre, precio y cantidad del equipo, y al final de `procesar_registro()` en la línea `continuar = input("\n¿Desea ingresar otro equipo? (s/n): ")`

Propósito dentro del programa: Capturar el texto que escribe el usuario por teclado para procesar los datos de los equipos o decidir si se repite el ciclo del programa.

6. INTEGRACIÓN DE LA LÓGICA DE V2.0

Decisión que se conserva: La verificación de si el subtotal supera los \$500 para aplicar un 5% de descuento (`if subtotal > 500:`), o la validación de que el precio y la cantidad sean mayores a cero (`if precio_unitario > 0 and cantidad > 0:`).

Función donde se encuentra la decisión: `calcular_totales_inventario()` para el descuento, y `procesar_registro()` para la validación de valores positivos.

Ciclo que se conserva: El ciclo `while` (`while continuar.lower() == "s":`) que permite seguir registrando más equipos electrónicos hasta que el usuario decida salir.

Función o parte del flujo donde se ejecuta el ciclo: Dentro de la función principal `procesar_registro()`.

7. PRUEBAS DE FUNCIONAMIENTO DE V3.0

Registra al menos dos pruebas realizadas después de modularizar el programa.

```
--- REGISTRO DE NUEVO EQUIPO ---
Ingrese el nombre del equipo electrónico: Laptop HP
Ingrese el precio del equipo electrónico ($): 600.00
Ingrese la cantidad de unidades recibidas: 1

¡Se aplicó un 5% de descuento por compra mayor a $500!

=====
RESUMEN DE REGISTRO DE INVENTARIO
=====
Equipo registrado:   Laptop HP
Cantidad de unidades: 1
Precio por unidad:   $600.00
Subtotal inicial:    $600.00
Descuento aplicado:  $30.00
Impuesto estimado (IVU): $65.55
Costo total abonado: $635.55
=====
```

```
--- REGISTRO DE NUEVO EQUIPO ---
Ingrese el nombre del equipo electrónico: Mouse
Ingrese el precio del equipo electrónico ($): 25.00
Ingrese la cantidad de unidades recibidas: 2

=====
RESUMEN DE REGISTRO DE INVENTARIO
=====
Equipo registrado:   Mouse
Cantidad de unidades: 2
Precio por unidad:   $25.00
Subtotal inicial:    $50.00
Descuento aplicado:  $0.00
Impuesto estimado (IVU): $5.75
Costo total abonado: $55.75
=====
```

```
--- REGISTRO DE NUEVO EQUIPO ---
Ingrese el nombre del equipo electrónico: Teclado
Ingrese el precio del equipo electrónico ($): -10.00
Ingrese la cantidad de unidades recibidas: 1

[ERROR] El precio y la cantidad deben ser valores mayores a cero.

¿Desea ingresar otro equipo? (s/n):
```

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido
1	Equipo: Laptop Precio: \$600.00 Cantidad: 1	Aplica el 5% de descuento por ser mayor a \$500. El subtotal final con el IVU del 11.5% debe ser \$635.55.	Se aplicó el descuento del 5%. Muestra correctamente el subtotal de \$570.00, IVU de \$65.55 y total de \$635.55.
2	Equipo: Mouse Precio: \$25.00 Cantidad: 2	No aplica descuento (subtotal \$50.00). Debe calcular el IVU (\$5.75) y mostrar un total de \$55.75.	Registró el equipo sin aplicar descuento. Muestra subtotal de \$50.00, IVU de \$5.75 y total abonado de \$55.75.
3	Equipo: Teclado Precio: -10.00 Cantidad: 1	Rechazar la entrada por tener un precio negativo y mostrar un mensaje de error.	Muestra el mensaje: [ERROR] El precio y la cantidad deben ser valores mayores a cero. y pide si desea continuar.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido
8. REFLEXIÓN DE EVOLUCIÓN			

¿Por qué v3.0 es más fácil de leer, reutilizar o mantener que v2.0? Explica en 4 a 6 oraciones y menciona al menos una función de tu programa como ejemplo.

La versión 3.0 es más fácil de leer porque el código está dividido en bloques ordenados con nombres claros en lugar de tener todo mezclado. También facilita el mantenimiento, ya que si cambia el porcentaje del impuesto, solo debo ajustar la función `calcular_totales_inventario()` sin tocar lo demás. Además, mejora la reutilización porque puedo llamar a una función como `solicitar_datos_equipo()` cuantas veces sea necesario sin repetir código. Esto evita duplicar instrucciones y hace que encontrar errores sea mucho más rápido. En resumen, el flujo principal del programa se entiende a simple vista y queda listo para añadir nuevas funciones en el futuro.

9. VALIDACIÓN

Etiquetas verificadas en el repositorio: ☐ v1.0 ☐ v2.0 ☐ v3.0

Archivo PDF en documentacion/: Documentacion_V1_V2_V3.pdf

URL del repositorio: _____

Nota: Esta plantilla no requiere diagramas de flujo ni diagramas modulares. La evidencia se concentra en la organización funcional del código, su documentación y las pruebas realizadas.